

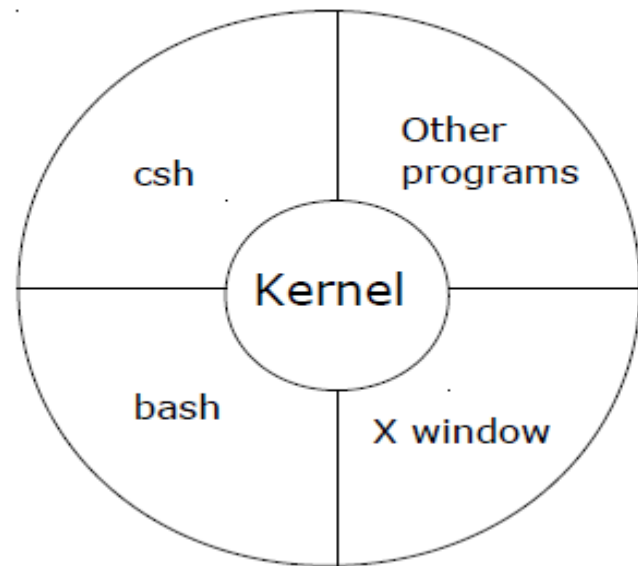
SHELL

SHELL

La forma que el usuario tiende de acceder a los servicios del kernel es mediante el Shell. Se puede considerar como el intérprete de comandos, que permite que los comandos y programas que se tecleen sean ejecutados.

SHELL

- Es la interfaz entre el usuario final y el Sistema Operativo.
- **No es el S.O.**
- Existen múltiples versiones y podemos averiguar cual tenemos instalada haciendo:
`% /bin/sh --version`



SHELL

- Interfaz entre el Sistema Operativo y el usuario
 - Dos tipos
 - Gráfica
 - Línea de Comandos
 - CLI (Command Line Interface)
 - intérprete de comandos, terminal, consola

es.wikipedia.org/wiki/Interfaz_gráfica_de_usuario

WIKIPEDIA
La enciclopedia libre

Portada
Portal de la comunidad
Actualidad
Cambios recientes
Páginas nuevas
Página aleatoria
Ayuda
Donaciones
Notificar un error

Imprimir/exportar
Crear un libro
Descargar como PDF
Versión para imprimir

Herramientas
Lo que enlaza aquí
Cambios en enlazadas
Subir un archivo
Páginas especiales
Enlace permanente
Información de la página
Elemento de

Artículo **Discusión**

Interfaz gráfica de usuario

La **interfaz gráfica de usuario**, conocida también como **GUI** (de que actúa de **interfaz de usuario**, utilizando un conjunto de imágenes y acciones disponibles en la interfaz. Su principal uso, consiste en permitir la comunicación con el **sistema operativo** de una máquina o computadora.

Habitualmente las acciones se realizan mediante manipulación directa en la computadora. Surge como evolución de las interfaces de **línea de comandos** y es pieza fundamental en un **entorno gráfico de usuario**. En los entornos de escritorio **Windows**, el **X-Window** de **GNU/Linux** o el **Mac OS**.

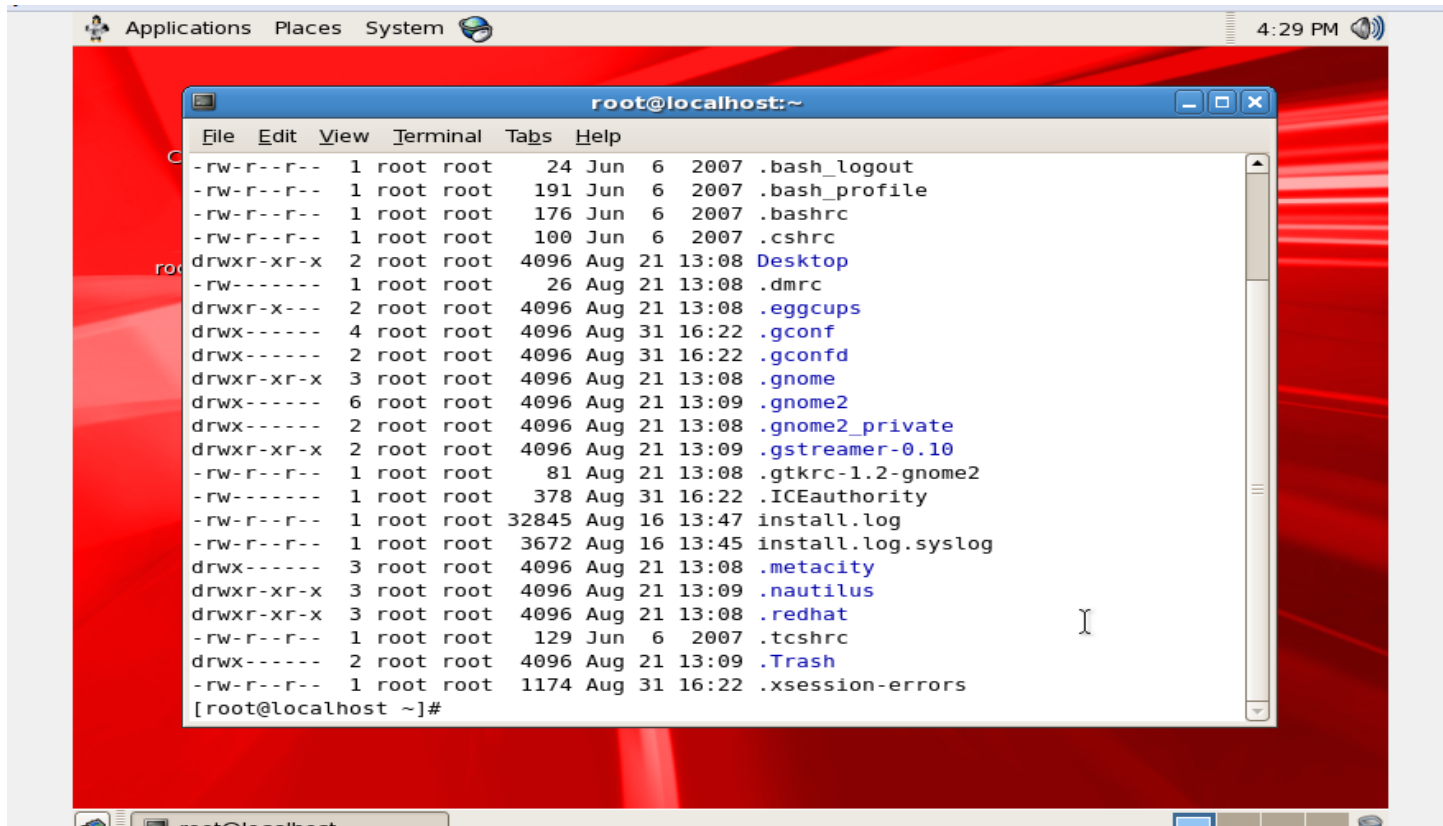
En el contexto del proceso de interacción **persona-ordenador**, la interfaz gráfica de usuario es el sistema interactivo que posibilita, a través del uso y la representación visual, la interacción entre el **sistema informático**.

Índice [ocultar]

- 1 Introducción
- 2 Precursores
- 3 Xerox Alto
- 4 Xerox Star 8010
- 5 Apple Lisa, Macintosh, Apple II GS
- 6 Workbench
- 7 GEOS
- 8 GEM

sistema de ventanas

Shell de Línea de Comandos



The screenshot shows a Linux desktop with a red background. A terminal window titled 'root@localhost:~' is open, displaying the output of a command that lists files and directories. The output is a table with columns for permissions, number of links, owner, group, size, date, time, and filename. The files listed include .bash_logout, .bash_profile, .bashrc, .cshrc, Desktop, .dmrc, .eggccups, .gconf, .gconfd, .gnome, .gnome2, .gnome2_private, .gststreamer-0.10, .gtkrc-1.2-gnome2, .ICEauthority, install.log, install.log.syslog, .metacity, .nautilus, .redhat, .tcshrc, .Trash, and .xsession-errors. The terminal window has a menu bar with File, Edit, View, Terminal, Tabs, and Help. The desktop has a menu bar with Applications, Places, and System. The system clock shows 4:29 PM.

```
root@localhost:~  
File Edit View Terminal Tabs Help  
-rw-r--r-- 1 root root 24 Jun 6 2007 .bash_logout  
-rw-r--r-- 1 root root 191 Jun 6 2007 .bash_profile  
-rw-r--r-- 1 root root 176 Jun 6 2007 .bashrc  
-rw-r--r-- 1 root root 100 Jun 6 2007 .cshrc  
drwxr-xr-x 2 root root 4096 Aug 21 13:08 Desktop  
-rw----- 1 root root 26 Aug 21 13:08 .dmrc  
drwxr-x-- 2 root root 4096 Aug 21 13:08 .eggccups  
drwx----- 4 root root 4096 Aug 31 16:22 .gconf  
drwx----- 2 root root 4096 Aug 31 16:22 .gconfd  
drwxr-xr-x 3 root root 4096 Aug 21 13:08 .gnome  
drwx----- 6 root root 4096 Aug 21 13:09 .gnome2  
drwx----- 2 root root 4096 Aug 21 13:08 .gnome2_private  
drwxr-xr-x 2 root root 4096 Aug 21 13:09 .gststreamer-0.10  
-rw-r--r-- 1 root root 81 Aug 21 13:08 .gtkrc-1.2-gnome2  
-rw----- 1 root root 378 Aug 31 16:22 .ICEauthority  
-rw-r--r-- 1 root root 32845 Aug 16 13:47 install.log  
-rw-r--r-- 1 root root 3672 Aug 16 13:45 install.log.syslog  
drwx----- 3 root root 4096 Aug 21 13:08 .metacity  
drwxr-xr-x 3 root root 4096 Aug 21 13:09 .nautilus  
drwxr-xr-x 3 root root 4096 Aug 21 13:08 .redhat  
-rw-r--r-- 1 root root 129 Jun 6 2007 .tcshrc  
drwx----- 2 root root 4096 Aug 21 13:09 .Trash  
-rw-r--r-- 1 root root 1174 Aug 31 16:22 .xsession-errors  
[root@localhost ~]#
```

Generalmente
con shell nos
referimos a
una CLI de
Unix

Gráfica vs Texto

Gráfica	Línea de Comandos
Sencilla Fácil uso	Ciertas operaciones son más rápidas
Usuario base	Usuario experto
Procesamiento de imagen, video, etc.	Transferencia o procesamiento de datos

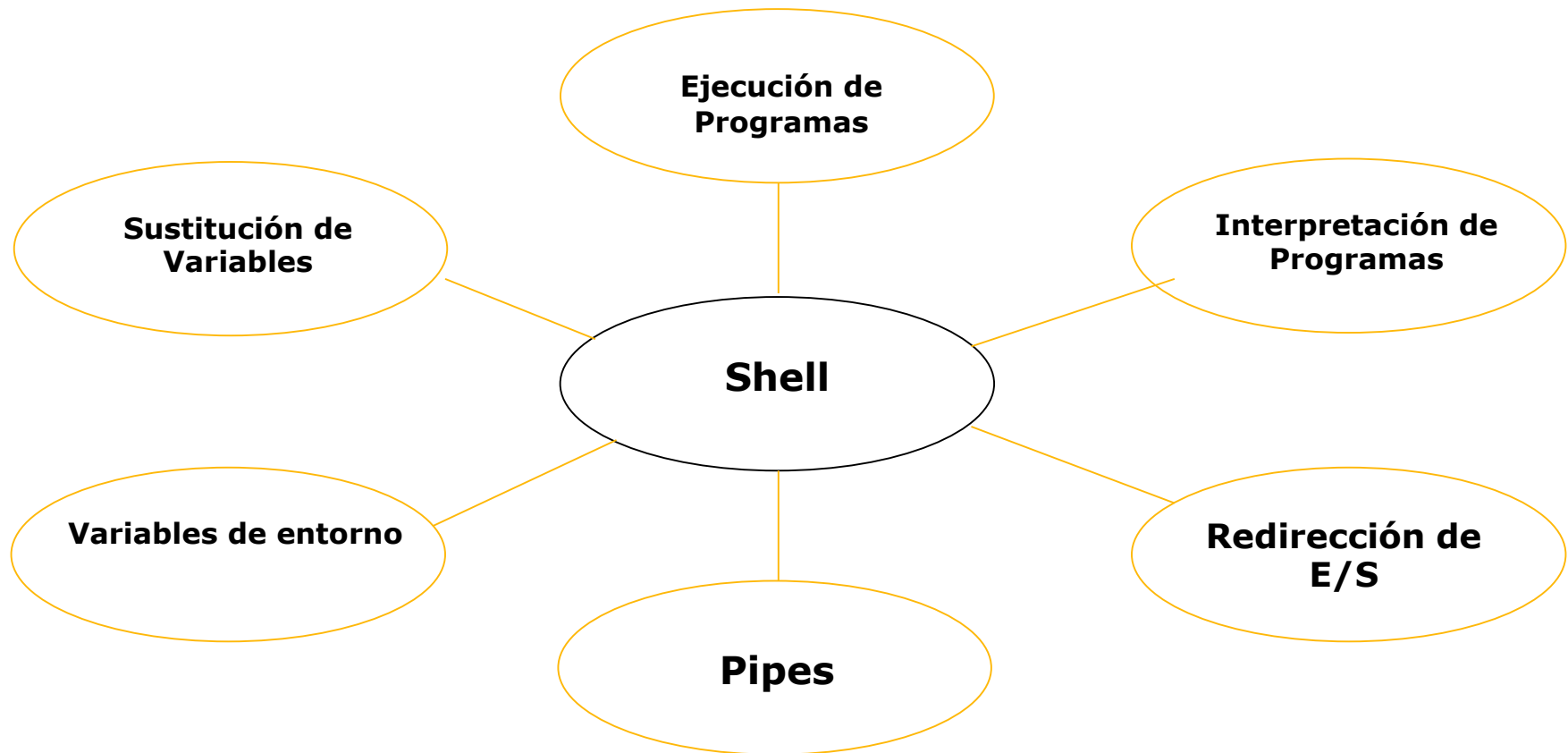
Shell de Línea de Comandos

- Interfaz de usuario basada en caracteres con el sistema operativo tipo UNIX (Linux, AIX, SunOS, etc.)
- Interacción con el usuario por línea de comandos
- Salida de los programas basada en texto
- Traduce las líneas de comandos en instrucciones del sistema operativo.

Shell de Línea de Comandos

- La shell NO es UNIX, sólo una interfaz con él
- UNIX es uno de los primeros SO en hacer la interfaz de usuario independiente del SO

Funciones del Shell



Funciones del Shell

Shell

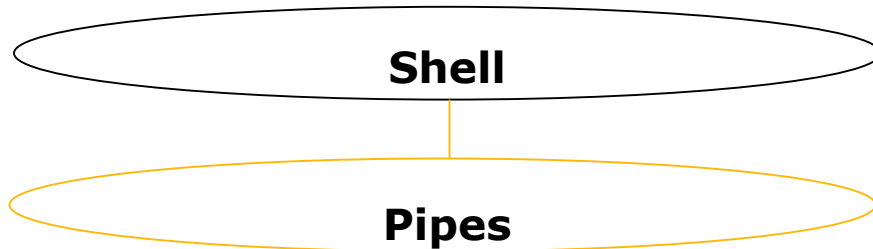
**Redirección de
E/S**

who > users

echo mario >> users

wc -l < users

Funciones del Shell



`ls -al | more`

`cat /etc/yum.conf | wc -l`
`ps aux | wc -l`
`ls | wc -l`

`head /var/log/messages | grep root`
`tail -f /var/log/messages | grep kernel`
`ls -l | awk '{print $3}'`

Pipes (o "tuberías") son un mecanismo que permite conectar la salida estándar de un comando a la entrada estándar de otro comando, usando el símbolo de barra vertical |. Esto te permite encadenar múltiples programas de manera secuencial, donde el resultado de un comando se pasa directamente como entrada al siguiente, simplificando tareas complejas

Este comando lista los archivos y directorios de forma detallada (`ls -l`). Luego, `awk` toma esa salida y, para cada línea, imprime solo el tercer campo (\$3), que suele ser el propietario.

SHELL

```
#!/bin/csh  
# A simple C-shell script  
echo -n "The date today is "  
date
```

Funciones del Shell

Tips

Shell

Variables de entorno

Las variables de entorno se utilizan para almacenar algunos valores que pueden ser utilizados por los scripts desde el shell.

El shell de bash tiene dos tipos de variables de entorno:

- Variables globales **
- Variables locales **

La variable local es visible solo en el shell donde definió, mientras que la variable global es visible para cualquier proceso en ejecución o que se ejecute desde el shell

`env`
`printenv`

vi editor - Que es, como usarlo, que comandos te recomiendo
<https://www.youtube.com/watch?v=iDsoWRdzAds>

Funciones del Shell

Tips

Shell

Variables de entorno

variable (temporal)

Para crear una variable temporal para tu sesión actual, usa export seguido del nombre de la variable y su valor:

Código

```
export MI_CONFIG="ruta/a/mi/config"
```

```
echo $MI_CONFIG.
```

variable persistente

Para que la variable se cargue cada vez que abras una nueva sesión de terminal, debes añadir el comando export a un archivo de configuración de tu shell, como ~/.bashrc (si usas Bash):

Abre el archivo con un editor de texto: vi ~/.bashrc

Añade la línea: export MI_CONFIG="ruta/a/mi/config"

Guarda y cierra el editor.

SHELLS y VARIABLES DE ENTORNO

echo \$SHELL.

```
$ env  
HOME=/home/user2  
PATH=/usr/bin:  
LOGNAME=user2  
HZ=100  
TERM=dtterm  
TZ=US/Mountain  
SHELL=/bin/csh  
MAIL=/var/mail/user2  
PWD=/home/user2  
USER=user2  
$
```

\$HOME: Indica el directorio principal de tu usuario.

\$USER: Muestra el nombre del usuario con el que iniciaste sesión.

\$PATH: Contiene una lista de directorios donde el sistema busca los comandos ejecutables.

\$SHELL: Indica el shell que estás utilizando actualmente.

```
C:\Documents and Settings>set
ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\wcastro.jeryta3p2L49835\AppData\Roaming
CommonProgramFiles=C:\Program Files\Common Files
CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
CommonProgramW6432=C:\Program Files\Common Files
COMPUTERNAME=JERYTA3P2L49835
ComSpec=C:\Windows\system32\cmd.exe
FP_NO_HOST_CHECK=NO
HOMEDRIVE=C:
HOMEPATH=\Users\wcastro.jeryta3p2L49835
LOCALAPPDATA=C:\Users\wcastro.jeryta3p2L49835\AppData\Local
LOGONSERVERS=\\JERYTA3P2L49835
NUMBER_OF_PROCESSORS=4
OS=Windows_NT
Path=C:\Program Files (x86)\PC Connectivity Solution\;C:\Program Files\Dell\DW W
LAN Card\;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\Sys
tem32\WindowsPowerShell\v1.0\;C:\Program Files\WIDCOMM\Bluetooth Software\;C:\Pr
ogram Files\WIDCOMM\Bluetooth Software\syswow64\;C:\Program Files (x86)\Intel\Op
enCL SDK\1.5\bin\x86;C:\Program Files (x86)\Intel\OpenCL SDK\1.5\bin\x64;C:\User
s\wcastro.jeryta3p2L49835\AppData\Local\Smartbar\Application\;
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE=AMD64
PROCESSOR_IDENTIFIER=Intel64 Family 6 Model 58 Stepping 9, GenuineIntel
PROCESSOR_LEVEL=6
PROCESSOR_REVISION=3a09
ProgramData=C:\ProgramData
ProgramFiles=C:\Program Files
ProgramFiles(x86)=C:\Program Files (x86)
ProgramW6432=C:\Program Files
PROMPT=$P$G
PSModulePath=C:\Windows\system32\WindowsPowerShell\v1.0\Modules\
PUBLIC=C:\Users\Public
SESSIONNAME=Console
SystemDrive=C:
SystemRoot=C:\Windows
TEMP=C:\Users\WCASTR~1\JER\AppData\Local\Temp
TMP=C:\Users\WCASTR~1\JER\AppData\Local\Temp
USERDOMAIN=jeryta3p2L49835
USERNAME=wcastro
USERPROFILE=C:\Users\wcastro.jeryta3p2L49835
VBOX_MSI_INSTALL_PATH=D:\Oracle\VirtualBox\
windir=C:\Windows
windows_tracing_flags=3
windows_tracing_logfile=C:\DUTDin\Tests\installpackage\cslogfile.log
C:\Documents and Settings>
```

SHELL

Volviendo la Shell es un interprete de comandos, como ya se ha dicho es el proceso que ejecuta la interpretación de la línea de comando. Lee de su entrada estándar y ejecuta el comando correspondiente a la línea alimentada.

Por lo tanto, un intérprete de comandos de Unix es tanto una interfaz de ejecución de órdenes y utilidades, como un lenguaje de programación, que admite crear nuevas órdenes – denominadas guiones o “shellscripts”–, utilizando combinaciones de mandatos y estructuras lógicas de control, que cuentan con características similares a las del sistema y que permiten que los usuarios y grupos de la máquina cuenten con un entorno personalizado

En Unix existen 2 familias principales de intérpretes de mandatos: los basados en el intérprete

de Bourne (BSH, KSH o BASH) y los basados en el intérprete C (CSH o TCSH).

SHELL

Para este caso Shell, haremos referencia especialmente a BASH (Bourne Again Shell) –evolución de BSH, con características de KSH y CSH–, ya que es el intérprete de mandatos más utilizado en Linux e incluye un completo lenguaje para programación estructurada y gran variedad de funciones internas.

SHELL

Cada shell, además de ejecutar las órdenes de LINUX, tiene sus propias órdenes y variables, lo que lo convierte en un lenguaje de programación.

La ventaja que presenta frente a otros lenguajes es su alta productividad -una tarea escrita en el lenguaje del shell suele tener menos código que si está escrita en un lenguaje como C.

SHELL

bash no es únicamente una excelente shell por línea de comandos ... También es un lenguaje de scripting en sí mismo El shell scripting permite utilizar las capacidades de la shell para automatizar multitud de tareas que, de otra forma, requerirían múltiples comandos introducidos de forma manual.

Lenguaje de programación vs. scripting: – Los lenguajes de programación son, en general, más potentes y mucho más rápidos que los lenguajes de scripting – Los lenguajes de programación comienzan desde el código fuente, que se compilan para crear los ejecutables (lo que permite que los programas sean fácilmente portables entre diferentes SO.

SHELL

- Un lenguaje de scripting también comienza por el código fuente, pero no se compila en un ejecutable
- En su lugar, un intérprete lee las instrucciones del archivo fuente y las ejecuta secuencialmente
- Los programas interpretados son, en general, más lentos que los compilados
- La principal ventaja reside en que el archivo de código fuente es fácilmente portable a cualquier sistema operativo

SHELL

```
#!/bin/csh  
# A simple C-shell script  
echo -n "The date today is "  
date
```

SHELL

Existen dos niveles de archivos de configuración:

General:

`/etc/profile`

`/etc/bashrc, /etc/cshrc, ...`

```
ls -l /etc/profile
file /etc/profile
cat /etc/profile
```

```
ls -l /etc/bashrc
file /etc/bashrc
cat /etc/bashrc
```

De usuario:

`~/.profile`

`~/.bashrc, ~/.cshrc`

```
cd /etc/skel
ls -la
more *
```

SHELL

Ciertos servicios se filtran basándose en el *shell* del usuario que lo invoca.

El Archivo `/etc/shells` indica qué ejecutables son *shells* válidos para el resto de servicios.

chsh -l

Shells de Programación

PERL

Tratamiento expresiones regulares interfaz con C y librerías de Utilidades

AWK

Procesador de campos

awk es un procesador de campos porque está diseñado para procesar datos como si fueran una serie de registros (líneas) y cada registro está compuesto por campos, que son partes de texto separadas por delimitadores.

TCL/TK

Componentes graficos

AWK

awk es un procesador de campos porque está diseñado para procesar datos como si fueran una serie de registros (líneas) y cada registro está compuesto por campos, que son partes de texto separadas por delimitadores.

```
df -h | awk '$5 > 30 {print $0}'
```

```
awk -F: '$3 >= 100 { print "Usuario: " $1 "\t Home: " $6 }'  
/etc/passwd
```

SHELL Y PROGRAMACION

- Brian W. Kernighan and Rob Pike, *El Entorno de Programación Unix*, Prentice Hall, 1987.
- Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Prentice Hall, 1978.
- Cameron Newham and Bill Rosenblatt, *Learning the bash Shell*, O'Reilly, October 1995.
- Kay A. Robbins and Steven Robbins, *UNIX Programación Práctica*, Prentice Hall, 1997.

Introducción a Shell Bash

- **Conceptos Claves**
- La shell bash se puede utilizar de modo interactivo o como un lenguaje de escritura de gran alcance.
- ·Al arranque Bash ejecuta comandos hallados en el archivo `~/.bashrc`, permitiéndole a los usuarios personalizar su shell.
- · La shell bash guarda el historial de las líneas de comando ejecutadas. La líneas de comando se pueden recuperar desde el historial.

La Shell Bash.

- En Linux, la shell es el programa más utilizado y creado para el proyecto “**GNU operating system**”..
- La shell es lo que usted ve cuando inicia sesión o cuando abre una terminal y lo que más usa para iniciar cada comando.
- Aunque hay una variedad de shells disponibles, pero todas siguen la misma conducta básica:
 - Escuchar los comandos del usuario,
 - Iniciar procesos como se especifica en los comandos
 - Informar los resultados al usuario.

La Shell Bash.

- La shell bash no sólo es de fácil uso para tareas sencillas, sino también tiene capacidades de gran alcance para facilitar tareas complejas o incluso hacerlas posibles.
- Como dato si vemos el manual del Bash este consta de 4500 líneas.

La Shell Bash.

- BASH incluye, entre otras, las siguientes características:
 - Edición y completación de comandos.
 - Histórico de comandos.
 - Control de procesos.
 - Funciones **SHELL** y alias.
 - Prompts personalizados
 - Operaciones aritméticas.

Shells interactivas vs. Scripts de shell

- La shell bash está diseñada para ser eficaz para dos tipos diferentes de uso.
- Shell Interactiva: esta característica de bash permiten a las personas escribir comandos de una manera más fácil y eficaz.
- Scripts de Shell: La shell bash también está diseñada para ser un lenguaje de escritura de gran alcance. Los scripts de la shell bash son programas pequeños escritos mediante la misma sintaxis que se utiliza en la línea de comandos.

Shells interactivas vs. Scripts de shell

- Los scripts de shell permiten a los usuarios automatizar las acciones repetidas al combinar una serie de comandos.
- A diferencia de las shells interactivas, los scripts de shell suelen ejecutar una serie de comandos de modo no interactivo y muchas de estas características de la shell bash proveen una programación lógica (tales como ramas y bucles) para escribir scripts sofisticados.

Shell de Inicio

- En la práctica, los usuarios a veces necesitan iniciar una shell de modo manual. Cada vez que alguien inicie sesión o abra una terminal, una shell se inicia automáticamente. Sin embargo, a veces los usuarios desearían ejecutar una shell diferente u otra instancia de la misma shell. Dado que la shell es sólo "otro programa", nuevas shells pueden iniciarse desde la shell existente. La nueva shell se denomina subshell de la shell original. Cuando se sale de la subshell, el control vuelve a la shell original

Shell de Inicio

■ Ejemplo

```
[madonna@station madonna]$ bash
[madonna@station madonna]$ ps
  PID TTY          TIME CMD
 9750 pts/5        00:00:00 bash
 9786 pts/5        00:00:00 bash
 9814 pts/5        00:00:00 ps
[madonna@station madonna]$ exit
exit
[madonna@station madonna]$
```

- Cuando inicia una subshell bash, las diferencias aparentes entre la subshell y la shell padre son mínimas y se debe tener cuidado de seguir el rastro de la shell en la que se encuentra.

Inicialización del Bash

- Cuando se inicializa la shell bash, este buscará en el directorio de inicio del usuario un archivo titulado **.bashrc**.
- El archivo se emplea para personalizar la shell bash.
- Cuando la shell inicia, los comandos listados en el archivo se ejecutan como si fueran escritos en la línea de comandos. Técnicamente, la shell bash "lee" el archivo.
- A continuación, el root edita su archivo ~/.bashrc agregándole el comando **cal**, para que tras el arranque la shell bash se presente un calendario del mes actual.

Inicialización del Bash

```
[madonna@station madonna]$ cat .bashrc
```

- **Ejemplo**
- Revisamos el contenido del archivo .bashrc

```
# .bashrc

# User specific aliases and functions

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

cal ❶
```

- ❶ La usuaria madonna agregó esta única línea. Las líneas restantes se encuentran en un archivo por defecto `~/ .bashrc` de un usuario.

Inicialización del Bash

- **Resultado**
- Ahora, cada vez que madonna inicia una shell bash (por ejemplo, iniciando en una consola virtual o abriendo otra ventana de terminal), se presenta un calendario.

```
[madonna@station madonna]$ bash
      August 2003
Su Mo Tu We Th Fr Sa
-*//                      1   2
 3   4   5   6   7   8   9
10  11  12  13  14  15  16
17  18  19  20  21  22  23
24  25  26  27  28  29  30
31
[madonna@station madonna]$ exit
```

Inicialización del Bash – Archivos de arranque del BASH

- Cuando el **BASH** es invocado de manera interactiva, primero lee y ejecuta comandos desde el archivo **/etc/profile**, si es que existe.
- Una vez leído este archivo, **BASH** ejecuta comandos del primer archivo que exista y que **BASH** pueda leer de entre los siguientes archivos: **~/.bash_profile**, **~/.bash_login**, y **~/.profile**, en ese orden.

Inicialización del Bash – Archivos de arranque del BASH

Cuando salimos de una “**sesión de login**”, **BASH** lee y ejecuta comandos desde el archivo **~/.bash_logout**, si es que este archivo existe.

Introducir Comandos

- Las shells interactivas repiten el ciclo de escuchar una línea de comandos, evalúan el comando solicitado, realizan todas las acciones solicitadas y muestran los resultados.
- La shell escucha al teclado de entrada y emplea la tecla de ENTER para reconocer el final de la entrada como en la siguiente ejecución del comando echo.

```
[madonna@station madonna]$ echo "hello world"  
hello world
```

Historial de Comandos

- Como conveniencia para los usuarios de shells interactivas, el comando shell bash mantiene el historial de cada uno de los comandos escritos por el usuario y ofrece una variedad de formas para hacer que los comandos desde este historial estén a su alcance.
- La forma más fácil de ver el historial actual es mediante el comando **history**.
- El comando history entrega un historial de los comandos previamente escritos, con cada entrada precedida por un "número de historial".

Historial de Comandos

- Ejemplo

```
[blondie@station blondie]$ history
```

```
1  ls -l /home/  
2  ls -ln /home/  
3  exit  
4  exit  
5  id
```

```
...
```

```
167 mv rhyme stuff/  
  
168 ls -Rli  
169 exit  
170  
171 exit  
172 history
```

Introducción a los Scripts de shell

- También le introdujeron a las características del historial del bash y le mostraron cómo referirse a comandos tecleados previamente.
- Sin embargo, aún falta una parte importante de la caja de herramientas del administrador del sistema: la escritura de scripts.
- ***Un script, en su forma más simple, es un texto con una lista de comandos en él. Los comandos se envían a través de un programa específico llamado intérprete, el cual ejecuta un comando a la vez.***

Introducción a los Scripts de shell

Este intérprete suele ser la shell bash (conocida como `/bin/bash` o `/bin/sh`) y cada comando es un comando común de Linux. Otros intérpretes permiten utilizar lenguajes de programación de gran alcance como **Perl, Python y Ruby**.

Datos Importantes:

La primera línea de su script debe especificar a qué intérprete se van a enviar las instrucciones. Esto se hace con una cadena especial llamada "**shebang**" (pronunciada "shuh-bang"), la cual se ve así: **#!**. A la shebang le sigue el nombre del intérprete para este script.

Ejemplo: para usar bash como su intérprete, se debería usar **#!/bin/sh** o **#!/bin/bash**.

Datos Importantes:

La mayoría de los scripts sólo usan **`#!/bin/sh`**.

Al referirse al intérprete como **`#!/bin/bash`** se habilitan otras características, pero se limita la compatibilidad del script con los sistemas antiguos de Unix y rara vez es necesario.

Datos Importantes:

Antes de ejecutar un script, usted debe habilitar el permiso "ejecutable" en él (de lo contrario, es sólo un archivo de texto).

El comando para esto es:

chmod u+x <scriptname>.

Le otorga (y sólo a usted) permiso para ejecutar este script justo como usted haría con otro comando.

Datos Importantes:

Si creó un script llamado **foo.sh** en su directorio de inicio y justo después tecleó **foo.sh** obtendría el mensaje de error "**no existe tal directorio o archivo**".

Esto se debe a que cuando teclea un comando hay una serie de directorios en donde Linux busca ese comando.

Datos Importantes:

Para resolver este problema tiene dos alternativas:

1. Usted usar **~/foo.sh** o **./foo.sh** (". " siempre se refiere al directorio actual).
2. Puede colocar el script en un directorio que sea parte de su RUTA. Los usuarios que no son root no tienen permiso para colocar archivos en la mayoría de estos directorios, pero todos los usuarios tienen un bin personal, al cual pueden escribir en su directorio de inicio. Por lo tanto, si **foo.sh** fuera movido a **~/bin** se podría ejecutar al teclear simplemente **foo.sh** en la línea de comandos.

Taller Clase

```
#!/bin/bash
```

```
echo -n 'Enter the First Number: '
```

```
read -r a
```

```
echo -n 'Enter the Second Number: '
```

```
read -r b
```

```
echo "$a + $b = $((a+b))"
```

```
#!/bin/bash
```

```
for c in 90 31 91 32 33 34 35 95 36 97; do
```

```
    echo -en "\r \e[${c}m LOVE \e[0m "
```

```
    sleep 1
```

```
done
```

```
#!/bin/bash
```

```
wc -l ./*
```

```
#!/bin/bash
```

```
echo -n "Enter The Number: "
```

```
read -r a
```

```
fact=1
```

```
while [ "$a" -ne 0 ]; do
```

```
    fact=$((fact * a))
```

```
    a=$((a - 1))
```

```
done
```

```
echo $fact
```

```
#!/bin/bash
x=0; y=1; i=2
while true; do
    i=$((i + 1))
    z=$((x + y))
    echo -n "$z "
    x=$y
    y=$z
    sleep .5
done
```

```
#!/bin/bash
echo "Hey what's Your First Name?"
read a
echo "welcome Mr./Mrs. $a, would you like to tell us,
Your Last Name"
read b
echo "Thanks Mr./Mrs. $a $b for telling us your name"
echo "*****"
echo "Mr./Mrs. $b, it's time to say you good bye"
```

```
#!/bin/bash
set -- *
for i; do
    echo "$i"
done
```

```
#!/bin/bash
echo -n "Enter Any Number: "
read -r n
i=1; c=1
while [ $i -le "$n" ]; do
    i=$((i + 1))
    r=$((n % i))
    [ $r -eq 0 ] && c=$((c + 1))
done

if [ $c -eq 2 ]; then
    echo "Prime"
else
    echo "Not Prime"
fi
```

```
#!/bin/bash
echo "Hello $USER"
echo "Hey i am $USER and will be telling you about the current
processes"
echo "Running processes List"
ps
```

TALLER ENTREGABLE